

Introduction to Entity Framework

In this article, I am going to give you a brief introduction to the Entity Framework. Before .NET 3.5 as a developer, we often used to write ADO.NET code to perform CRUD operations with the underlying database. For this, we need to create a Connection Object with the database, then Open the Connection, Create the Command Object and execute the Command using Data Reader or Data Adapter. And then we create DataSet or DataTables to store the data in memory to perform different types of Operations on the Data as per the business requirements. Actually, this is a Time-Consuming, and Error-Prone Process. Microsoft has provided a framework called “Entity Framework” to automate all these database-related activities for our application and for this to work, we just need to provide the necessary details to the Entity Framework.

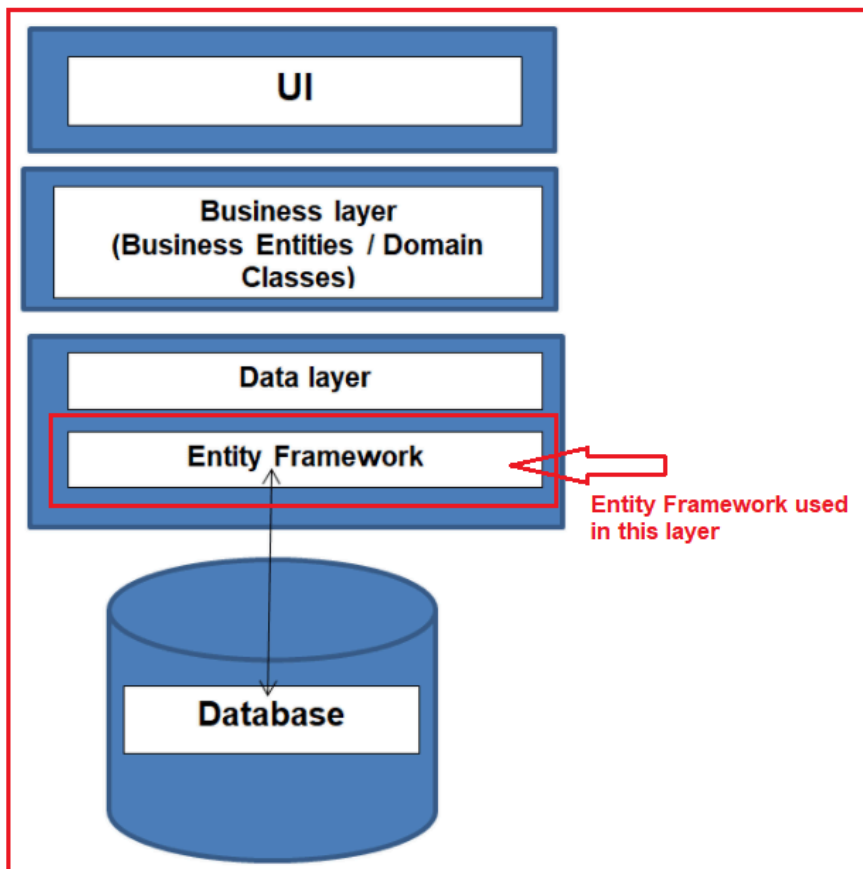
What is the Entity Framework?

Entity Framework is an **Open-Source Object-Relational Mapping (ORM) Framework** for .NET applications that enables .NET developers to work with relational data using domain-specific objects without focusing on the underlying database tables and columns where actually the data is stored. That means the Entity Framework eliminates the need for writing the data-access code that developers usually need to write.

Official Definition: Entity Framework is an object-relational mapper (O/RM) that enables .NET developers to work with a database using .NET objects. It eliminates the need for most of the data-access code that developers usually need to write.

What is an Object-Relational Mapping Framework?

Object Relational Mapping framework automatically creates classes based on database tables, and vice versa is also true, that is, it can also automatically generate the necessary SQL to create database tables based on classes. The following diagram illustrates where the Entity Framework fits into our application.



The above diagram shows that the Entity Framework fits between the Data Layer and the database. It saves the data in the database which are stored in the properties of the business entities (domain classes) and also retrieves the data from the database and converts it to business entities objects automatically.

What Entity Framework Can do for us?

Let's understand what entity framework can do for us with an example. Assume we have the following 2 tables (Departments and Employees).

Departments			Employees					
ID	Name	Location	ID	Name	Email	Gender	Salary	DepartmentId
1	IT	Mumbai	1	Mark	Mark@g.com	Male	60000	1
2	HR	Delhi	2	Steve	Steve@g.com	Male	45000	3
3	Sales	Hyderabad	3	Pam	Pam@g.com	Female	60000	1
			4	Sara	Sara@g.com	Female	345000	3
			5	Ben	Ben@g.com	Male	70000	1
			6	Philip	Philip@g.com	Male	45000	2
			7	Mary	Mary@g.com	Female	30000	2
			8	Valarie	Valarie@g.com	Female	35000	3
			9	John	John@g.com	Male	80000	1

We want to display the data from both tables (Departments and Employees) in a console application as shown below.

```
Department = IT, Location = Mumbai
    Name = Mark, Email = Mark@g.com, Gender = Male, salary = 60000
    Name = Pam, Email = Pam@g.com, Gender = Female, salary = 60000
    Name = Ben, Email = Ben@g.com, Gender = Male, salary = 70000
    Name = John, Email = John@g.com, Gender = Male, salary = 80000

Department = HR, Location = Delhi
    Name = Philip, Email = Philip@g.com, Gender = Male, salary = 45000
    Name = Mary, Email = Mary@g.com, Gender = Female, salary = 30000

Department = Sales, Location = Hyderabad
    Name = Steve, Email = Steve@g.com, Gender = Male, salary = 45000
    Name = Sara, Email = Sara@g.com, Gender = Female, salary = 345000
    Name = Valarie, Email = Valarie@g.com, Gender = Female, salary = 35000
```

To achieve this

1. We need to create Department and Employee classes
2. We need to write ADO.NET code to retrieve the data from the database

3. Once the data is retrieved we need to create Department and Employee objects and populate them with the retrieved data.

But Entity Framework can do all of the above automatically for us if we provide the necessary database schema to the Entity Framework. Let us understand this step by step how to achieve the same using Entity Framework.

Creating the Database Schema to Understand Entity Framework

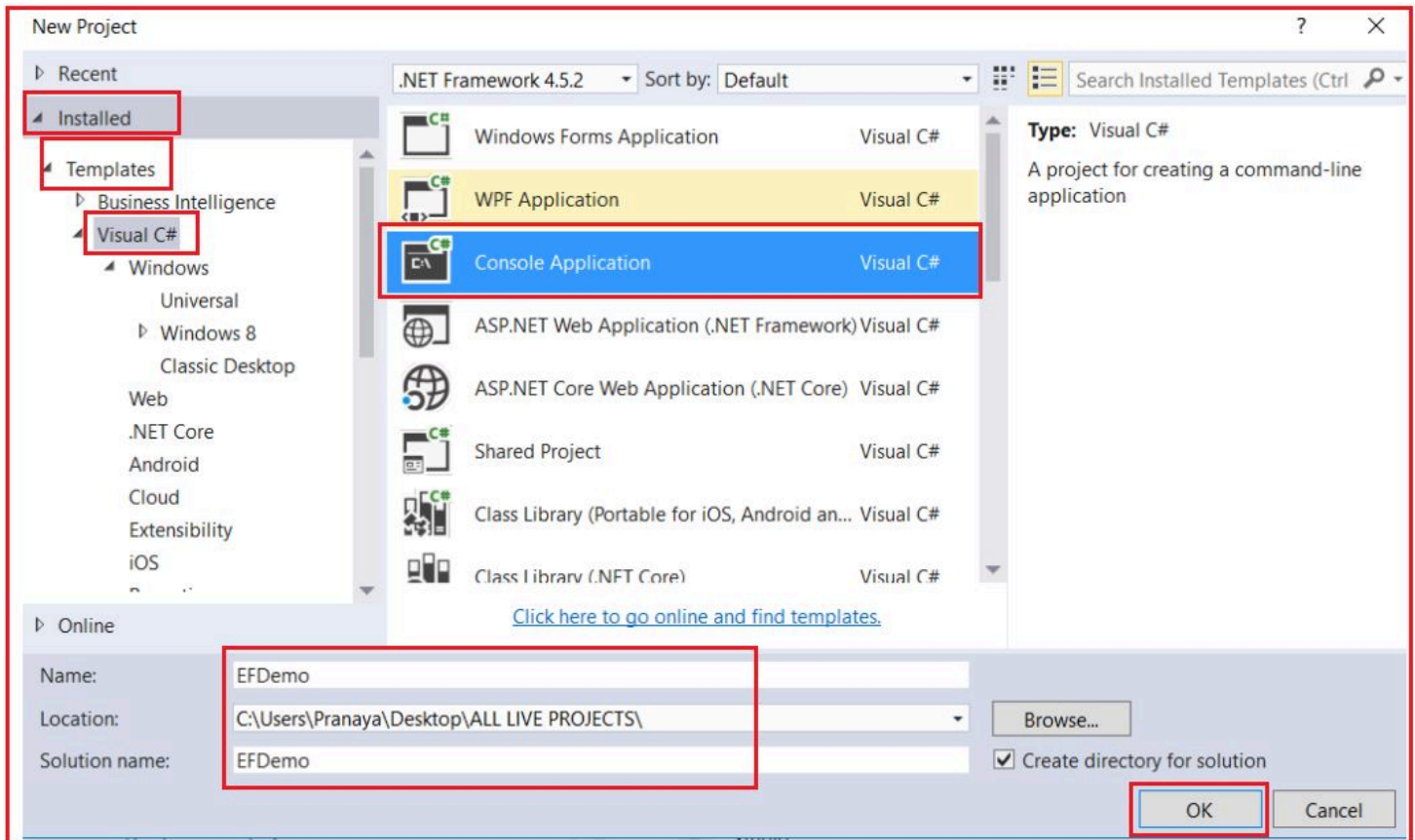
Please use the below SQL script to create the database **EF_Demo_DB**, and tables **Departments** and **Employees**, and populate the tables with sample data.

```
-- Create the EF_Demo_DB database.
CREATE DATABASE EF_Demo_DB;
GO
-- Use EF_Demo_DB database
USE EF_Demo_DB;
GO
-- Create Departments table
CREATE TABLE Departments
(
    ID INT PRIMARY KEY IDENTITY(1,1),
    Name VARCHAR(50),
    Location VARCHAR(50)
)
Go
-- Create Employees table.
CREATE TABLE Employees
(
    ID INT PRIMARY KEY IDENTITY(1,1),
    Name VARCHAR(50),
    Email VARCHAR(50),
    Gender VARCHAR(50),
    Salary INT,
    DepartmentId INT FOREIGN KEY REFERENCES Departments(ID)
)
Go
--Populate the Departments table with some test data
INSERT INTO Departments VALUES ('IT', 'Mumbai')
INSERT INTO Departments VALUES ('HR', 'Delhi')
INSERT INTO Departments VALUES ('Sales', 'Hyderabad')
Go
--Populate the Employees table with some test data
INSERT INTO Employees VALUES ('Mark', 'Mark@g.com', 'Male', 60000, 1)
INSERT INTO Employees VALUES ('Steve', 'Steve@g.com', 'Male', 45000, 3)
INSERT INTO Employees VALUES ('Pam', 'Pam@g.com', 'Female', 60000, 1)
INSERT INTO Employees VALUES ('Sara', 'Sara@g.com', 'Female', 345000, 3)
INSERT INTO Employees VALUES ('Ben', 'Ben@g.com', 'Male', 70000, 1)
INSERT INTO Employees VALUES ('Philip', 'Philip@g.com', 'Male', 45000, 2)
INSERT INTO Employees VALUES ('Mary', 'Mary@g.com', 'Female', 30000, 2)
INSERT INTO Employees VALUES ('Valarie', 'Valarie@g.com', 'Female', 35000, 3)
INSERT INTO Employees VALUES ('John', 'John@g.com', 'Male', 80000, 1)
```

Go

Create a Console Application

Once the database is ready, in the next step, create a new “**Console Application**” with the name **EFDemo** as shown in the below image.



Example without using Entity Framework:

Let us first see the example without using Entity Framework and then we will see the same example using Entity Framework. In the below example, we are using ADO.NET to interact with the database to fetch the data and display it in the console.

```
using System;
using System.Data;
using System.Data.SqlClient;
namespace EFDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                string ConnectionString = @"data source=LAPTOP-ICA2LCQL\SQLEXPRESS; database=EF_Demo_DB; integrated security=SSPI";
```

```

        using (SqlConnection connection = new
SqlConnection(connectionString))
        {
            //Create the SqlDataAdapter instance by specifying the
command text and connection object
            SqlDataAdapter dataAdapter = new SqlDataAdapter("SELECT *
FROM Departments; SELECT * FROM Employees;", connection);
            //Creating DataSet Object
            DataSet dataSet = new DataSet();
            //Filling the DataSet using the Fill Method of
SqlDataAdapter object
            dataAdapter.Fill(dataSet);
            DataTable DepartmentDataTable = dataSet.Tables[0];
            DataTable EmployeeDataTable = dataSet.Tables[1];
            foreach (DataRow row in DepartmentDataTable.Rows)
            {
                Console.WriteLine($"  Department = {row["Name"]},
Location = {row["Location"]}");
                DataView dataView = EmployeeDataTable.DefaultView;
                //Applying Single Filter
                dataView.RowFilter = "DepartmentId = " + row["ID"];
                foreach (DataRowView rowView in dataView)
                {
                    DataRow emp = rowView.Row;
                    Console.WriteLine($"  \t Name = {emp["Name"]},
Email = {emp["Email"]}, Gender = {emp["Gender"]}, salary = {emp["Salary"]}");
                }
            }
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Exception Occurred: {ex.Message}");
    }
    Console.ReadKey();
}
}
}
}
}

```

Now, run the application and you will get the output as expected as shown in the below image.

```
Department = IT, Location = Mumbai
    Name = Mark, Email = Mark@g.com, Gender = Male, salary = 60000
    Name = Pam, Email = Pam@g.com, Gender = Female, salary = 60000
    Name = Ben, Email = Ben@g.com, Gender = Male, salary = 70000
    Name = John, Email = John@g.com, Gender = Male, salary = 80000

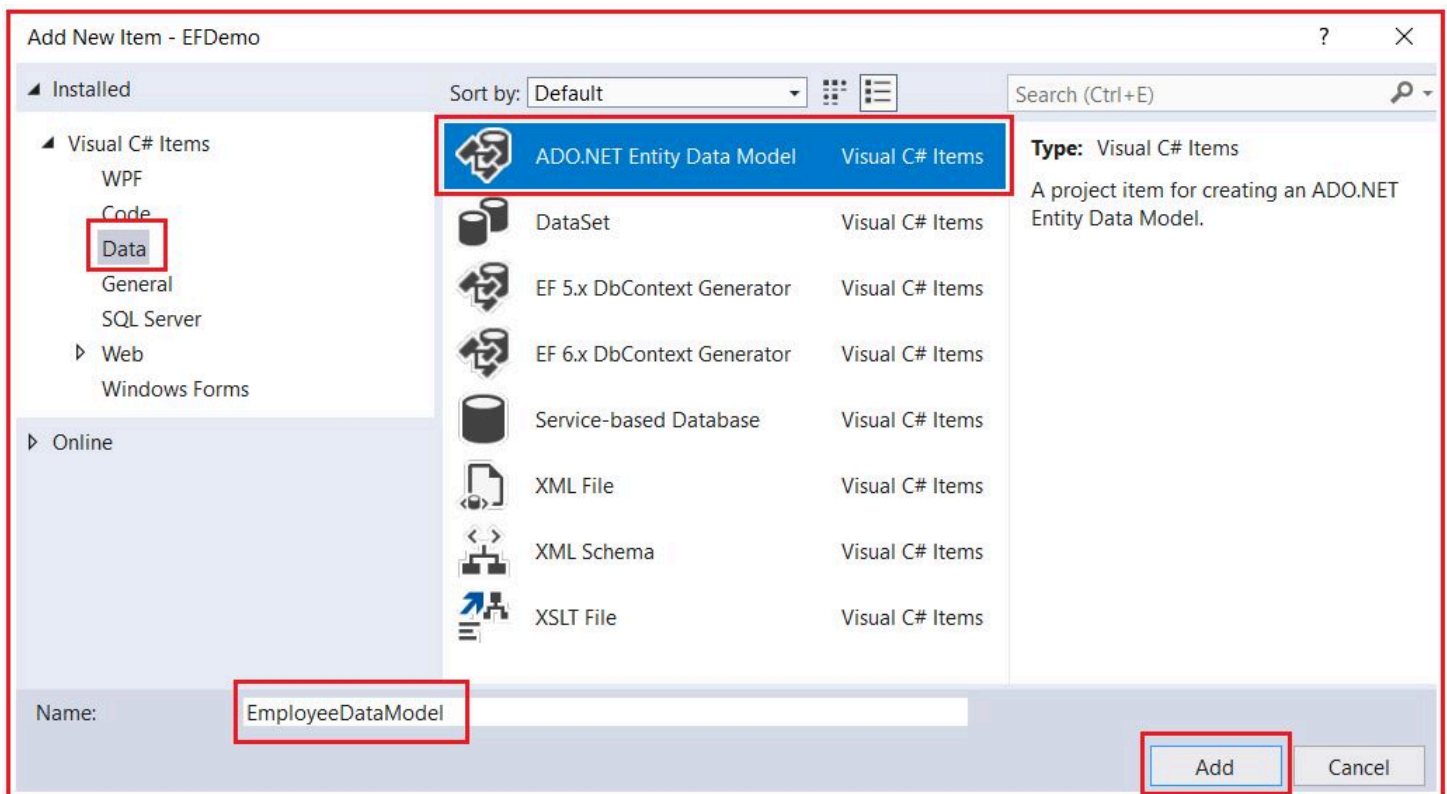
Department = HR, Location = Delhi
    Name = Philip, Email = Philip@g.com, Gender = Male, salary = 45000
    Name = Mary, Email = Mary@g.com, Gender = Female, salary = 30000

Department = Sales, Location = Hyderabad
    Name = Steve, Email = Steve@g.com, Gender = Male, salary = 45000
    Name = Sara, Email = Sara@g.com, Gender = Female, salary = 345000
    Name = Valarie, Email = Valarie@g.com, Gender = Female, salary = 35000
```

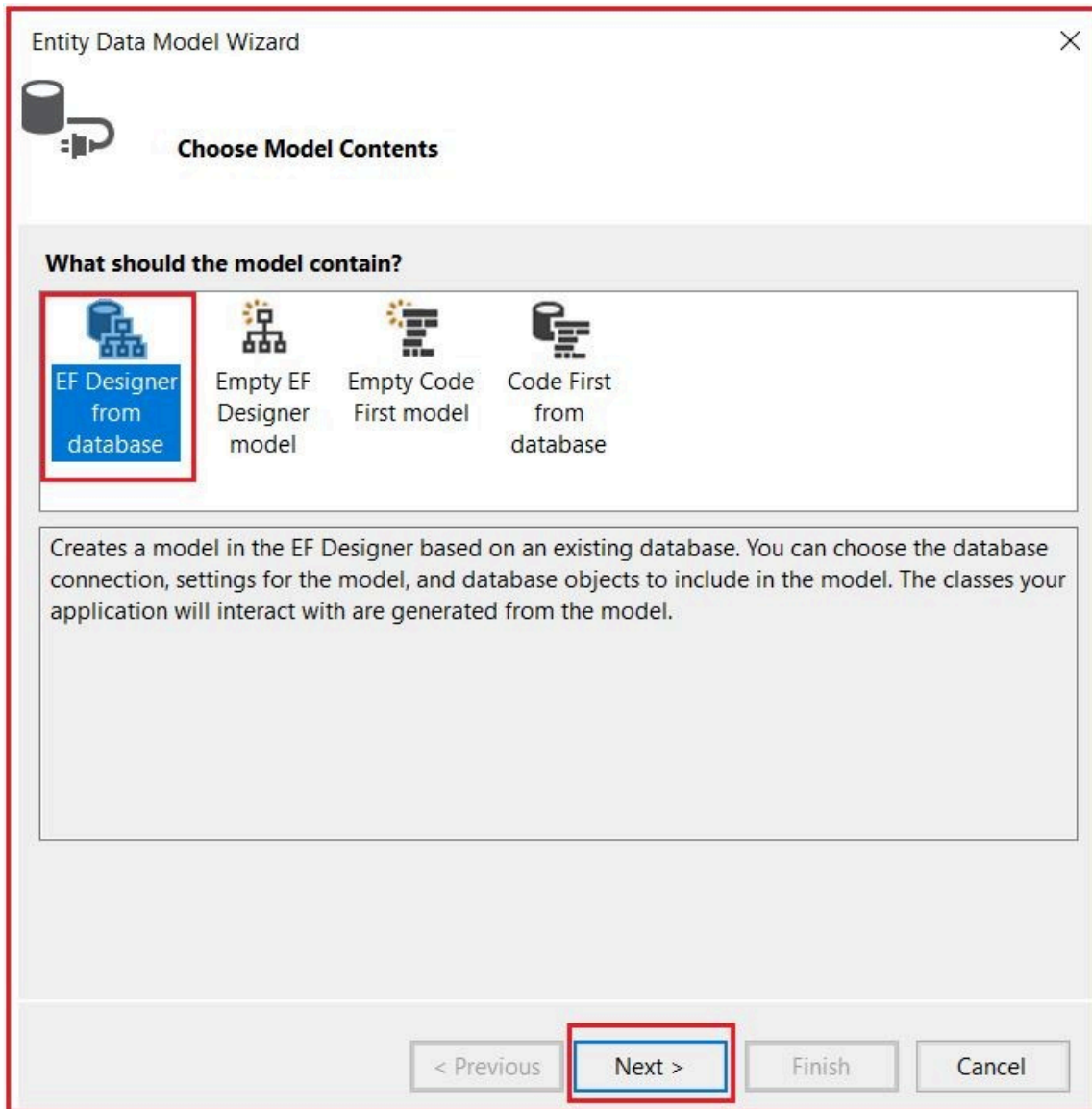
Here, you can observe we have created the connection object, created the data adapter object, and created Dataset, Data Tables, and DataView objects. And here we are also writing the SQL Commands to fetch the data from the database tables and fill the dataset. We can avoid all these ADO.NET Related things if we use Entity Framework. Let us proceed and see how we can rewrite the same example using Entity Framework using C# language.

Adding ADO.NET Entity Data Model

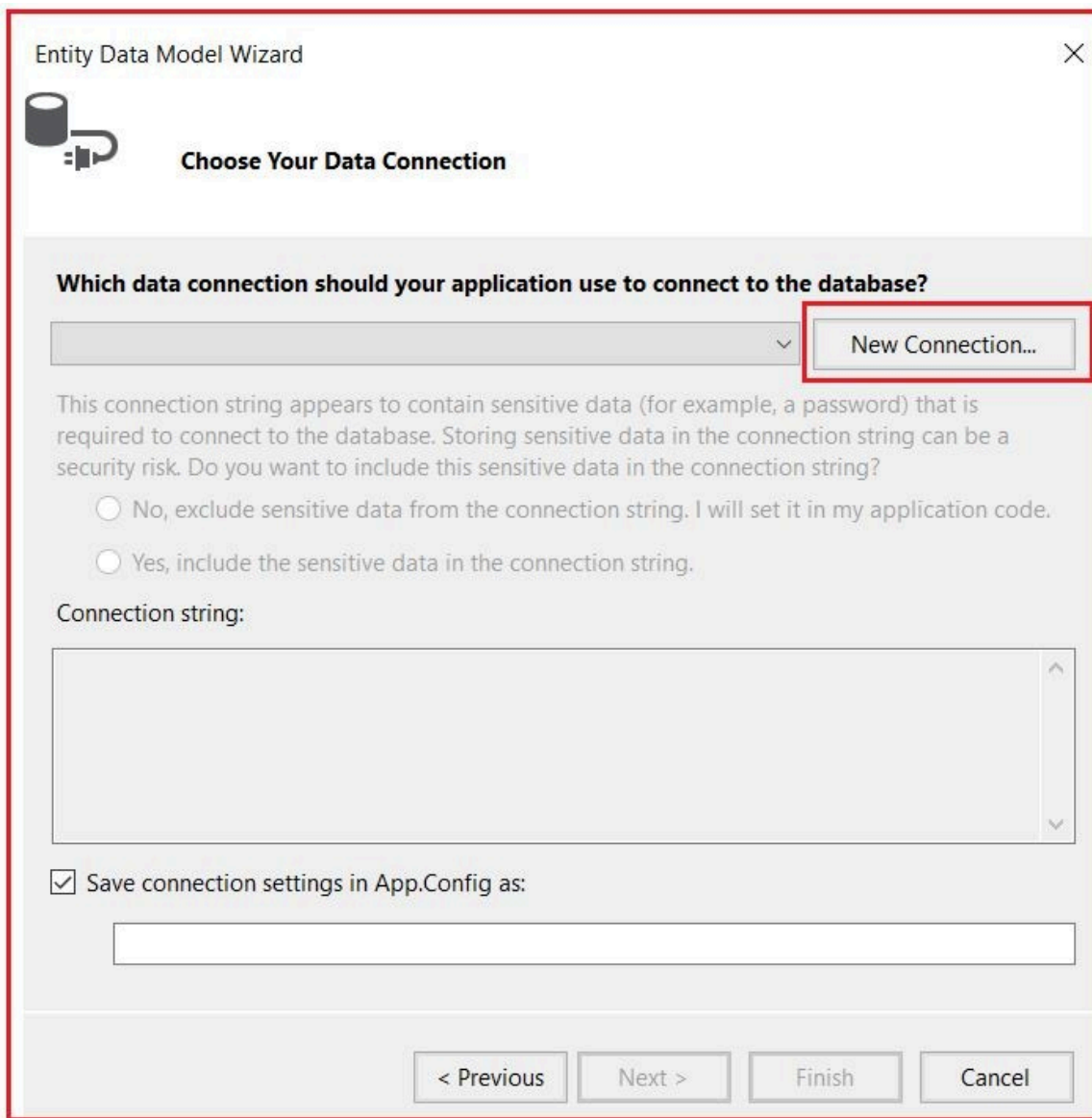
To use Entity Framework, we need to add ADO.NET Entity Data Model. To do so, Right-click on the project in solution explorer and select **Add => New Item**, and then click on **Data** Section from the left side panel and choose **ADO.NET Entity Data Model** from the middle panel, Change the name from Model1 to **EmployeeDataModel** and click on the Add button as shown in the below image.



Once you click on the **Add** button, it will open the Choose Data Model wizard as shown below. From this window select **“EF Designer from database”** and click on the **Next** button as shown in the below image. Here, select the **“EF Designer from database”** option as we are going to use the Entity Framework Database First Approach.



Once you click on the **Next** button, It will open choose your data connection wizard as shown below. From this window click on the **New Connection** button as shown in the below image.



The image shows a screenshot of the 'Entity Data Model Wizard' window, specifically the 'Choose Your Data Connection' step. The window has a title bar with the text 'Entity Data Model Wizard' and a close button. Below the title bar is a logo consisting of a cylinder and a plug, followed by the text 'Choose Your Data Connection'. The main content area is titled 'Which data connection should your application use to connect to the database?'. It features a dropdown menu with a downward arrow, and a button labeled 'New Connection...' is highlighted with a red rectangle. Below this, there is a paragraph of text: 'This connection string appears to contain sensitive data (for example, a password) that is required to connect to the database. Storing sensitive data in the connection string can be a security risk. Do you want to include this sensitive data in the connection string?'. There are two radio buttons: 'No, exclude sensitive data from the connection string. I will set it in my application code.' and 'Yes, include the sensitive data in the connection string.'. Below the radio buttons is a label 'Connection string:' followed by a large text area. At the bottom left, there is a checked checkbox labeled 'Save connection settings in App.Config as:' followed by a text input field. At the bottom right, there are four buttons: '< Previous', 'Next >', 'Finish', and 'Cancel'.

Once you click on the **New Connection** button, it will open a popup for providing the database details as shown below. **From this connection properties screen,**

1. Select **"Microsoft SQL Server"** as the Data Source, and the **".Net Framework Data Provider for SQL Server"** option from the **"Data provider"** drop-down list. Click **Continue**.
2. On the "Connection Properties" screen, specify SQL Server Name.
3. Specify the Authentication you want to use.
4. Select the database from the "Select or Enter a Database Name" drop-down list.
5. Finally "Test Connection" and click "OK"

Connection Properties

Enter information to connect to the selected data source or click "Change" to choose a different data source and/or provider.

Data source:
Microsoft SQL Server (SqlClient) Change...

Server name:
LAPTOP-ICA2LCQL\SQLEXPRESS Refresh

Log on to the server
Authentication: Windows Authentication

Connect to a database

☒ Select or enter a database name:
EF_Demo_DB

☐ Attach a database file:
Browse...

Logical name:

Advanced...

Test Connection OK Cancel

Once you click on the **OK** button, you will be back on to the **"Choose Your Data Connection"** window as shown below. Make sure the **"Save entity connection settings in App.Config as"** checkbox is selected and change the name of the connection string to **EF_Demo_DBEntities** and then Click on the **"Next"** button as shown in the below image.

Entity Data Model Wizard

 **Choose Your Data Connection**

Which data connection should your application use to connect to the database?

laptop-ica2lcql\sqlexpress.EF_Demo_DB.dbo

New Connection...

This connection string appears to contain sensitive data (for example, a password) that is required to connect to the database. Storing sensitive data in the connection string can be a security risk. Do you want to include this sensitive data in the connection string?

☐ No, exclude sensitive data from the connection string. I will set it in my application code.

☐ Yes, include the sensitive data in the connection string.

Connection string:

```
metadata=res://*/EmployeeDataModel.csdl|res://*/EmployeeDataModel.ssdl|
res://*/EmployeeDataModel.msl;provider=System.Data.SqlClient;provider connection
string="data source=LAPTOP-ICA2LCQL\SQLEXPRESS;initial catalog=EF_Demo_DB;integrated
security=True;MultipleActiveResultSets=True;App=EntityFramework"
```

☒ Save connection settings in App.Config as:

EF_Demo_DBEntities

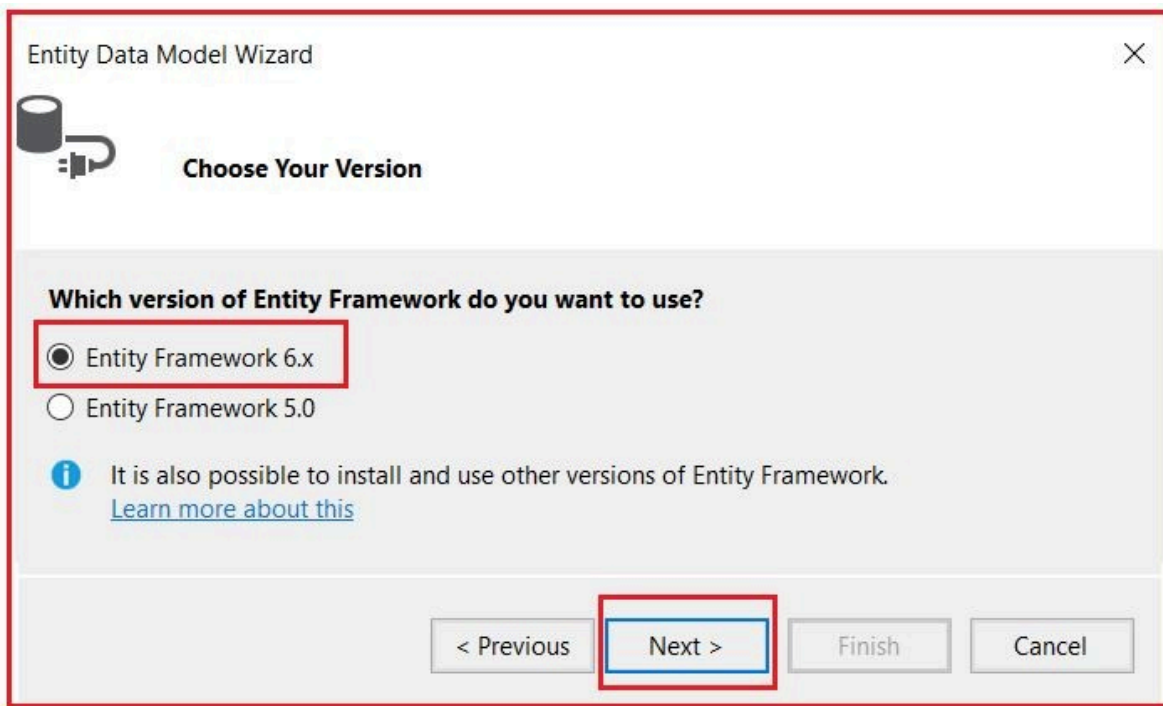
< Previous

Next >

Finish

Cancel

Once you click on the **Next** button, it will open a popup asking you to choose the Entity Framework version as shown below. From this window, select **Entity Framework 6.x** and click on the Next button as shown in the below image.



Once you click on the Next button, it will open Choose Your Database Objects as shown below. From the below window, select the “**Departments**” and “**Employees**” tables. Change the Model Namespace to “**EmployeeModel**” and click on the “**Finish**” button as shown in the below image.

Entity Data Model Wizard

Choose Your Database Objects and Settings

Which database objects do you want to include in your model?

- ☒ Tables
 - ☒ dbo
 - ☒ Departments
 - ☒ Employees
- ☐ Views
- ☐ Stored Procedures and Functions

☒ Pluralize or singularize generated object names

☒ Include foreign key columns in the model

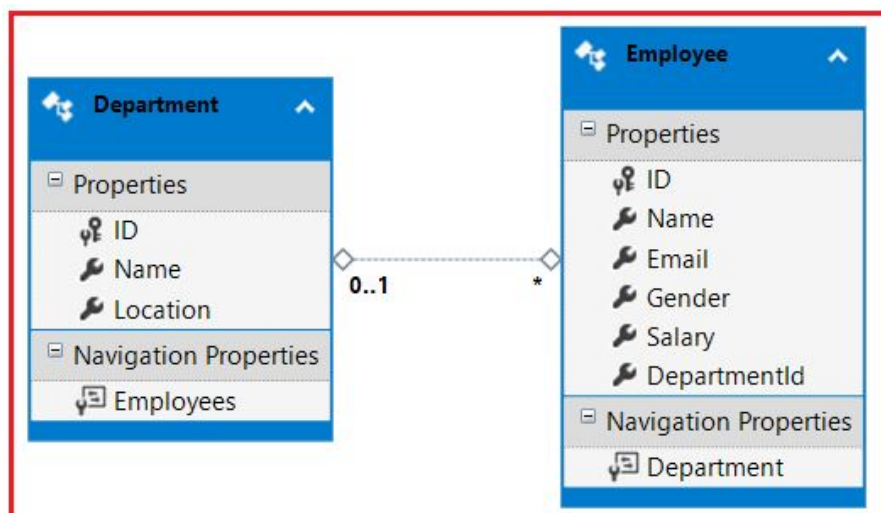
☐ Import selected stored procedures and functions into the entity model

Model Namespace:

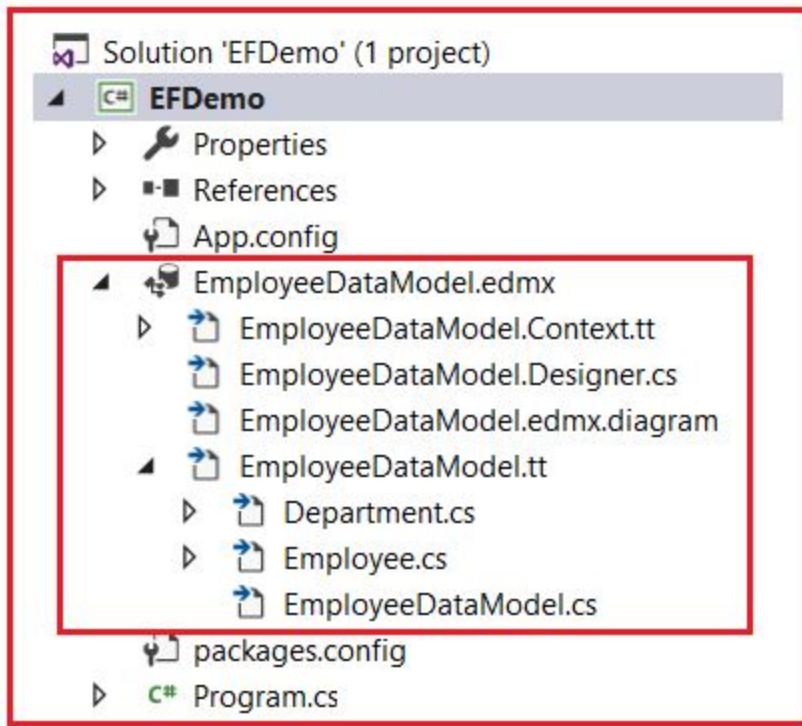
EmployeeModel

< Previous Next > **Finish** Cancel

Once you click on the **"Finish"** button, it should generate the **EmployeeDataModel.EDMX** file.



Following is the structure of the EDMX file.



The Entity Framework Create **Employee** and **Department** classes automatically based on the **Employees** and **Departments** Tables. Tables are mapped to classes and columns are mapped to class properties.

Using Entity Framework in C#:

Now copy and paste the following code in the main method of the program class as shown below. Here, in this article, I am not going to explain the code. Here, I just want to show how to use Entity Framework. From our next article onwards, I am going to explain everything in detail.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace EFDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities DBEntities = new EF_Demo_DBEntities())
            {
                List<Department> listDepartments =
                DBEntities.Departments.ToList();
                Console.WriteLine();
                foreach (Department dept in listDepartments)
                {
                    Console.WriteLine("  Department = {0}, Location = {1}",
                    dept.Name, dept.Location);
                    foreach (Employee emp in dept.Employees)
                    {
                        Console.WriteLine("\t Name = {0}, Email = {1}, Gender
                        = {2}, salary = {3}",
```

```

        emp.Name, emp.Email, emp.Gender, emp.Salary);
    }
    Console.WriteLine();
}
Console.ReadKey();
}
}
}
}
}

```

Run the application and notice that Departments and Employees data are displayed as expected as shown in the below image. We have achieved all this without writing a single line of ADO.NET code. If this is not clear at this moment how it works, don't worry we will discuss everything in detail in our upcoming articles.

```

Department = IT, Location = Mumbai
    Name = Mark, Email = Mark@g.com, Gender = Male, salary = 60000
    Name = Pam, Email = Pam@g.com, Gender = Female, salary = 60000
    Name = Ben, Email = Ben@g.com, Gender = Male, salary = 70000
    Name = John, Email = John@g.com, Gender = Male, salary = 80000

Department = HR, Location = Delhi
    Name = Philip, Email = Philip@g.com, Gender = Male, salary = 45000
    Name = Mary, Email = Mary@g.com, Gender = Female, salary = 30000

Department = Sales, Location = Hyderabad
    Name = Steve, Email = Steve@g.com, Gender = Male, salary = 45000
    Name = Sara, Email = Sara@g.com, Gender = Female, salary = 345000
    Name = Valarie, Email = Valarie@g.com, Gender = Female, salary = 35000

```

Why do we need to use Entity Framework in .NET Applications?

Entity Framework is an ORM Tool and ORMs Tools are basically used to increase the developer's productivity by reducing the redundant task of doing CRUD operation against a database in a .NET Application.

1. Entity Framework can generate the necessary database commands for doing the database CRUD Operation i.e. can generate SELECT, INSERT, UPDATE and DELETE commands for us.
2. While working with Entity Framework, we can perform different types of operations on the domain objects (basically classes representing database tables) using LINQ to entities.
3. Entity Framework will generate and execute the SQL Command in the database and then store the results in the instances of your domain objects so that you can do different types of operations on the data.

Entity Framework Features:

1. **Cross-platform:** EF Core is a cross-platform framework which means it can run on Windows, Linux, and Mac operating systems.
2. **Modeling:** EF (Entity Framework) creates an EDM (Entity Data Model) based on POCO (Plain Old CLR Object) entities with get/set properties of different data types. It uses this model also when querying or saving entity data to the underlying database.

3. **Querying:** EF allows us to use LINQ queries (C#/VB.NET) to retrieve data from the underlying database. The database provider will translate these LINQ queries to the database-specific query language (e.g. SQL for a relational database). EF also allows us to execute raw SQL queries directly to the database.
4. **Change Tracking:** EF keeps track of changes that occurred to instances of our entities (Property values) that need to be submitted to the database.
5. **Saving:** EF executes INSERT, UPDATE, and DELETE commands to the database based on the changes that occurred to our entities when we call the SaveChanges() method. EF also provides the asynchronous SaveChangesAsync() method.
6. **Concurrency:** EF uses Optimistic Concurrency by default to protect against overwriting changes made by another user since data was fetched from the database.
7. **Transactions:** EF performs automatic transaction management while querying or saving data. It also provides options to customize transaction management.
8. **Caching:** EF includes the first level of caching out of the box. So, repeated querying will return data from the cache instead of hitting the database.
9. **Built-in Conventions:** EF follows conventions over the configuration programming pattern, and includes a set of default rules which automatically configure the EF model.
10. **Configurations:** EF allows us to configure the EF model by using data annotation attributes or Fluent API to override default conventions.

In the next article, I am going to discuss the [Architecture of the Entity Framework](#). Here, in this article, I try to explain **Introduction to Entity Framework** with an example. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article